

Figure 6.1 A typical LangChain operation chain where it processes data sequentially through a `toUpperCase` function, then a `VowelCount` function, then via the LLM, and finally via the `StringOutputParser` to parse the output.

Files: simple-chain.openai.js, simple-chain.google.js

The code for this example project is in the ch06 folder. To run these examples, first navigate into the ch06 directory from your project root (e.g., if you were in the ch05 folder, you would run `cd ../ch06`):

```
$ npm install
$ node simple-chain.openai.js
```

If you are using Google AI, there is a separate file named agents.google.js in the same folder:

```
$ npm install
$ node simple-chain.google.js
```

You'll need an OpenAI API key to access their REST API or a Google API key for Google services. The appendix offers instructions on how to set up and create a secret key for OpenAI and other providers:

```
// .env

OPENAI_API_KEY=<SECRET_KEY>
GEMINI_API_KEY=<YOUR_GEMINI_API_KEY>
```

For details on setting up and creating API keys, refer to the appendix.

The following listing shows the code for this simple chain.

Listing 6.1 Example using runnable chains

```
import { StringOutputParser } from "@langchain/core/output_parsers";
import { ChatPromptTemplate } from "@langchain/core/prompts";
import { RunnableLambda } from "@langchain/core/runnables";
```